Voyage-AI — Comprehensive Project Report

Document Reference: `docs/reports/project-report.pdf` / `docs/reports/project-report.md`
Project Title: Voyage-AI: All-in-One AI-Powered Travel Discovery & Itinerary Companion
Document Version: 0.2.1 (Phase 2 Verified Baseline)
Author: Senior Software Architect, Systems Analyst & QA Lead
Publication Date: October 25, 2026
Status: Implementation-Verified Project Report

---

1. Executive Summary

Voyage-AI is an intelligent travel planning platform engineered to unify destination exploration, curated accommodation discovery, culinary trails, and day-by-day itinerary management into a single, cohesive application. Developed as a modern decoupled modular monolith, the system combines a React 18 / TypeScript single-page application with a high-performance Spring Boot 4.1.1 (Java 17 LTS) REST backend and a MySQL 8.0 relational persistence foundation.

This comprehensive technical report documents the project's architecture, functional capabilities, engineering methodology, empirical test outcomes, and future development trajectory. All statements reflect the **actual verified codebase** as of October 25, 2026:
- **Phase 0 (Repository & Runtime Scaffolding):** Fully completed and verified. Build scripts, container orchestration, and health monitoring endpoints (`/api/v1/health` , `/actuator/health`) are operational.
- **Phase 1 (Design System & Discovery Prototype):** Fully completed and verified. A tactile, editorial user interface inspired by Emil Kowalski design principles is active, offering multi-criteria filtering across curated Indian destinations (Jaipur, Varanasi, Munnar, Hampi) with 5-tab detail inspection modals and transparent budget tiers.
- **Phase 2 (Backend, Persistence & Verified Integrations):** Fully completed and verified.
- Single canonical MySQL 8.0 database **`voyage`** on Docker port 3307 with host port 3306 preserved untouched.
- Flyway versioned migrations (`V0` through `V4`) creating `users` , `destinations` , `travel_listings` , `trips` , `itinerary_days` , and `itinerary_items` .
- Stateless JWT Bearer token authentication, BCrypt password hashing, and role-based access control (`ROLE_USER` , `ROLE_ADMIN`).
- Persistent user profiles (`/api/v1/users/me`) with preference management and avatar synchronization.
- Full-catalog server-side pagination (`page` , `size=12`) with multi-attribute search filtering and pagination response headers.
- Real Wikimedia destination data ingestion pipeline consuming English Wikipedia summaries and MediaWiki Action API for Creative Commons image licensing (`CC-BY-SA 4.0` , `CC0` , `Public Domain`) and photographer credits.
- Administrative authorization guard (`@PreAuthorize("hasRole('ADMIN')")`) on batch ingestion.
- User-scoped multi-day trip planning with multi-tenant ownership enforcement (`403 Forbidden` on cross-user access).
- Live Google Gemini AI integration (`gemini-3.5-flash-lite`) verified end-to-end via browser automation.
- Live Open-Meteo weather integration with coordinate caching and CC-BY 4.0 provenance.
- Single-command local developer launcher `start.bat` ensuring non-destructive service reuse.
- **Verification Outcomes:** 100% of executable tests passed cleanly during verification (128 / 128 tests: 83 backend, 11 frontend, 34 Playwright browser & integration). Detailed in POST_IMPLEMENTATION_VERIFICATION.md.

2. Problem Statement & Strategic Objectives

2.1 The Crisis of Modern Travel Planning

Contemporary travel planning suffers from severe fragmentation, user distrust, and deceptive digital practices:
1. **Tool Fragmentation:** Independent leisure travelers are forced to navigate 5 to 8 disconnected platforms—using search engines for discovery, review aggregators for dining, online travel agencies (OTAs) for lodging, map tools for transit routing, and spreadsheets for scheduling.
2. **Synthetic & Hallucinated Data:** AI-based travel assistants frequently hallucinate non-existent attractions, closed venues, or fictitious pricing, resulting in ruined trips and loss of user confidence.
3. **Deceptive Urgency & Dark Patterns:** Commercial booking platforms exploit high-pressure dark patterns and obscure hidden service fees until checkout.
4. **Sterile Admin Interfaces:** Existing travel software often resembles sterile corporate admin dashboards or cluttered coupon aggregators rather than inspiring travel publications.

2.2 Core Project Objectives

- **Unified Discovery-to-Itinerary Pipeline:** Provide an intuitive bridge transitioning users from inspiring destination discovery directly into a structured, editable day-by-day schedule.
- **Honest Data Provenance & Zero-Fabrication:** Eliminate synthetic numbers. All travel facts, pricing benchmarks, and weather conditions must be stamped with source provider attribution, retrieval timestamps, and explicit verification status. Missing data must be displayed honestly as unverified or unavailable.
- **Warm Modernism & Tactile Interaction:** Deliver a high-craft, editorial visual aesthetic with tactile UI feedback (`:active:scale(0.97)`), WCAG 2.2 AA accessibility, and restrained animations under 250ms.
- **Robust Offline-First Engineering:** Support deterministic local development with isolated database guarantees and zero dependencies on paid third-party APIs.

---

3. Project Scope & Deliverables Matrix

```
+-----------------------------------------------------------------------------------+
|                                PROJECT SCOPE MATRIX                               |
+-----------------------------------------------------------------------------------+
|                                                                                   |
|  [IMPLEMENTED & VERIFIED SCOPE (Phases 0, 1, and 2 Baseline)]                     |
|  - Decoupled client-server architecture with Spring Boot 4.1.1 and React 18 Vite SPA    |
|  - Custom health monitoring endpoints (/api/v1/health, /actuator/health)          |
|  - Emil Kowalski tactile design system (Warm modernism, custom tokens, active states)    |
|  - Full-catalog destination exploration with server-side pagination (12 items/page)     |
|  - Multi-attribute filtering (Region, Travel Style, Budget Tier, Text Search)     |
|  - 5-Tab Destination Inspection Modal (Overview, Stays, Dining, POIs, Budget, Media)    |
|  - Per-image photographer credits and clickable Creative Commons license links in UI    |
|  - Single Canonical MySQL 8.0 database (`voyage`) on Docker port 3307 (host untouched)   |
|  - Flyway migrations V0-V4 (baseline, schema, authentic seeds, indexes, roles/licensing)|
|  - Stateless JWT Bearer authentication filter chain, BCrypt hashing, and role checks   |
|  - User profile management (/api/v1/users/me) with preference storage and role security |
|  - Admin ingestion pipeline (/api/v1/admin/destinations/import) guarded by ROLE_ADMIN   |
|  - Real Wikipedia summary and Wikimedia Commons Action API media extraction       |
|  - User-scoped Trip CRUD APIs with multi-tenant ownership enforcement (403 Forbidden)   |
|  - Auto-generated itinerary calendar days matching trip start/end date ranges     |
|  - Live Google Gemini AI integration (verified browser round-trip: "New Delhi")        |
|  - Open-Meteo weather integration service with WMO codes and CC-BY 4.0 provenance DTOs  |
|  - Windows development orchestrator (`start.bat`) with automated service dependency waits|
|  - Centralized automated testing infrastructure (128/128 tests passing, 100% pass rate) |
|                                                                                   |
|  [PLANNED FUTURE SCOPE (Phases 3, 4, and 7)]                                      |
|  - Phase 3: Interactive Leaflet map widget with OpenStreetMap tile layer          |
|  - Phase 4: OSRM transit distance matrix with Haversine direct distance fallback       |
|  - Phase 7: Production cloud container deployment (Render / Railway / AWS ECS)    |
|                                                                                   |
|  [EXPLICIT EXCLUSIONS (By Architectural Design)]                                   |
|  - Live payment gateway processing or flight/hotel transaction bookings           |
|  - Commercial GDS integrations requiring paid developer access                    |
|  - Automated web scraping violating third-party terms of service                  |
+-----------------------------------------------------------------------------------+
```

---

4. System Overview & Technology Stack

4.1 High-Level Architecture

Voyage-AI employs a decoupled **modular monolith** architecture designed for operational simplicity and maintainability:

- **Presentation Layer (`frontend/`):** React 18 SPA built with TypeScript and Vite. It consumes backend REST endpoints via standard `fetch` and renders editorial views styled with Tailwind CSS.
- **Application Layer (`backend/`):** Java 17 LTS Spring Boot 4.1.1 service operating on port `8081`. It manages cross-origin access (`CorsConfig.java`), uniform error envelopes (`GlobalExceptionHandler.java`), Spring Security 6 stateless JWT auth, and isolated external service adapters.
- **Data Layer (`infra/` & `backend/src/main/resources/db/`):** MySQL 8.0 relational database with single canonical schema `voyage` managed through Flyway migrations V0-V4.
- **Test Automation (`tests/` & submodules):** Multi-tiered test execution combining MockMvc, isolated H2 in-memory databases, Vitest, and Java Playwright (128 tests verified passing).

4.2 Verified Technology Summary

Layer	Primary Technology	Version	Location	Role in System
Language & SDK	Java JDK	17.0.17 LTS	Host Workstation	Backend compilation & execution
Backend Framework	Spring Boot	4.1.1	`backend/pom.xml`	Web container, REST dispatching, Actuator
Security	Spring Security / JWT	6.x	`backend/pom.xml`	Stateless JWT Bearer auth, BCrypt, Roles
Data Access	Spring Data JPA / Hibernate	4.1.1 / 7.4.5	`backend/pom.xml`	Relational ORM abstraction (6 domain entities)
Migrations	Flyway Core & MySQL	10.x / 12.0.0	`backend/pom.xml`	Database schema versioning (V0 through V4)
Database	MySQL Server	8.0.44	`infra/docker-compose.yml`	Canonical persistent store (`voyage` on port 3307)
AI Provider	Google Gemini API	`gemini-3.5-flash-lite`	`backend/src/main/java/.../ai/`	Conversational travel assistant (live verified)
Weather Provider	Open-Meteo API	Forecast v1	`backend/src/main/java/.../weather/`	Meteorological forecast data (live verified)
Media Provider	Wikimedia Commons API	Action API & REST v1	`backend/src/main/java/.../ingestion/`	Real destination summaries & CC-licensed media
Frontend Engine	Node.js / npm	v24.13.1 / 11.8.0	Host Workstation	Build runtime and package management
UI Library	React / TypeScript	18.3.1 / 5.6.2	`frontend/package.json`	Declarative components and static typing
Styling	Tailwind CSS	3.4.14	`frontend/package.json`	Design tokens, responsive layouts, transitions
End-to-End Tests	Java Playwright	1.48.0	`tests/pom.xml`	Cross-browser UI automation & API verification
Local Orchestrator	Windows Batch	1.0.0	`start.bat`	One-click service orchestration and dependency waits

---

5. Verification & Quality Outcomes

During the verification phase on October 25, 2026, the complete automated test suite was executed across all layers:
- **Backend Core & Integration Suite:** 83 passed, 0 failures, 0 errors, 0 skipped.
- **Frontend Unit Suite (Vitest):** 11 passed, 0 failures, 0 skipped.
- **Browser Automation & API Suite (Playwright):** 34 passed, 0 failures, 0 skipped.
- **Overall Result: 128 passed, 0 failures, 0 skipped (100% pass rate)**.
- **Live Provider Testing:** Real Google Gemini prompt round-trip returned `"The capital of India is **New Delhi**."` in browser Playwright testing; real Open-Meteo returned live forecast metrics for New Delhi (`29.3°C`, `Clear sky`).
- **Database Safety:** Programmatic isolation assertions in `DatabaseFlywayIntegrationTest` confirmed zero connections to host MySQL port 3306 or unapproved schemas.